

VisionBridge — Offline OCR Implementation: Tonight’s Step-by-Step Guide

Smart Reading (“Read Text”) only · fine-tuned PP-OCRv5 → ONNX → ONNX Runtime on Android · written 17 Sep 2026 against your current visionbridge repository.

Use it like this: do Step 0 → paste Section 11 into Antigravity → work through Steps 1–15 (Antigravity codes, you run the training notebook) → tick Section 10.

1 · Tonight’s target

- A **fine-tuned OCR model running on the phone** replaces only the text-extraction step of Smart Reading. Every other VisionBridge feature stays exactly as it is.
- Flow: CameraX capture → on-device text detection → fine-tuned on-device recognition → text + confidence → existing Smart Reading UI + TTS.
- At runtime the feature needs: **no internet, no cloud OCR, no Gemini, no Supabase call** — only local ONNX Runtime inference on the phone’s CPU.
- Report honestly what ships: the **detector** is the official pretrained model, the **recognizer** is your fine-tuned model (or “pretrained” if training does not finish).

2 · Fastest practical implementation path

[YOU] = you do it · [AG] = Antigravity does it from the Section 11 prompt. Steps 9–12 run on the official pretrained model *while* Step 6 trains — afterwards you swap one file.

#	Step	What to do (tool → output)	Done when
0	Checkpoint [YOU] 2 min	Your repo has many uncommitted and untracked files on main (including the new MedSafe files). In the Android Studio terminal run the commands below.	git status is clean and you are on feature/offline-ocr.
1	Inspect Smart Reading [AG] 10 min	Paste Section 11 into Antigravity (planning mode). It searches the repo before editing anything.	You get an implementation map naming SmartReadingScreen → ViewModel → Repository → Api → Edge Function. Reply “go” only if it matches.
2	Find the capture flow [AG]	Capture is already CameraX ImageCapture (one JPEG + rotation) in SmartReadingScreen.kt — reuse it; no new camera code.	The map says “no camera changes”.
3	Find the current OCR [AG]	Today the text comes from Gemini through SmartReadingRepository.extractText(), returning ReadingExtraction.	The map lists that one function as the single switch point.
4	Choose the model [YOU] 1 min	Take Section 4 as given: PP-OCRv5 mobile detector + en_PP-OCRv5_mobile_rec recognizer.	Nothing to install yet.
5	Get the base models [AG] 5 min	Antigravity creates ocr-training/ (Colab notebook + scripts) and downloads the two official ONNX files so Android work can start immediately.	The notebook exists and app/src/main/assets/ocr/ holds the detector + pretrained recognizer.
6	Dataset + fine-tune [YOU] 60–90 min	Take 40–60 phone photos (Section 5). Open the notebook in Colab, Runtime → GPU, run cells: synthetic data → pre-label photos → fix labels → baseline → train.	output/vb_en_rec/best_accuracy.pdparams exists and beats the baseline on your real photos.
7	Export to ONNX [YOU] 5 min	The notebook runs tools/export_model.py, then paddlex --paddle2onnx.	onnx/vb_en_rec/inference.onnx (~8 MB) and “ONNX vs Paddle: same text on ≥ 98% of val crops”.
8	Optimize (optional) [YOU] 10 min	Try INT8 static quantization in the notebook. Keep it only if accuracy drops ≤ 1 point and the phone is measurably faster.	The size/accuracy table is printed and you picked FP32 or INT8 on evidence.
9	Add model files [AG]	Models + dictionary go to app/src/main/assets/ocr/; add the ONNX Runtime dependency.	Gradle sync succeeds and the APK contains assets/ocr/.
10	Integrate inference [AG] 60–90 min	New ocr/ package (engine, detector, recognizer, CTC decoder) called only from SmartReadingRepository.	Unit tests pass and the offline instrumented test finds “PARACETAMOL” in androidTest/assets/reading-sample.jpg.
11	Connect UI + TTS [AG]	Return the same ReadingExtraction; the existing ViewModel, 0.70 “hard to read” banner and TextToSpeechManager do the rest.	The phone speaks the captured text.
12	Build the APK [AG/YOU]	.\gradlew.bat :app:assembleDebug → install on the phone.	BUILD SUCCESSFUL and the app launches.
13	Airplane-mode test [YOU] 10 min	Airplane mode ON, Wi-Fi OFF, data OFF → Read Text → capture a page.	Text is read aloud; logcat shows VB-Offline0cr timings and no Smart Reading network call.
14	Benchmark [AG/YOU] 15 min	Swap in the fine-tuned recognizer, re-test, and record size, time, memory and accuracy (Section 8).	Every number is measured on your phone — none estimated.
15	Regression check [YOU] 15 min	With the network back on: login, home, Gemini features, Live, Volunteer Help, MedSafe, TTS.	Section 10 is fully ticked.

```
git add -A
git commit -m "Checkpoint before offline OCR"
git checkout -b feature/offline-ocr
```

If git says index.lock already exists, close other git tools and delete .git\index.lock, then retry.

3 · Exact tools to install / use

Tool	Why you need it tonight	Install / link
Git + Android Studio (latest stable), phone with USB debugging	Checkpoint, build, install, Logcat, Network Inspector, Profiler	developer.android.com/studio
Antigravity	Writes the Android code and the training scripts	already installed
Google Colab — GPU runtime (free T4 when available). Backup: Kaggle Notebooks	Trains the model without installing CUDA or Python on Windows	colab.research.google.com
PaddlePaddle GPU 3.3.0	Training engine	<code>pip install paddlepaddle-gpu==3.3.0 -i https://www.paddlepaddle.org.cn/packages/stable/cu126/</code>
PaddleOCR v3.7.0 (repo + pip package)	Fine-tune config, training/export scripts, pre-labeling of your photos	<code>git clone --depth 1 --branch v3.7.0 https://github.com/PaddlePaddle/PaddleOCR.git ; pip install -r PaddleOCR/requirements.txt ; pip install paddleocr==3.7.0</code>
Paddle2ONNX plugin (PaddleX CLI ships with paddleocr)	Converts the fine-tuned model to .onnx	<code>paddlex --install paddle2onnx</code>
onnx + onnxruntime (Python 1.30)	Checks the exported model, optional INT8, golden reference outputs	<code>pip install onnx onnxruntime</code>
huggingface_hub	Downloads the official ONNX detector / recognizer	<code>pip install -U huggingface_hub</code>
ONNX Runtime for Android 1.30.0	Runs both models on the phone	<code>implementation("com.microsoft.onnxruntime:onnxruntime-android:1.30.0")</code>
Netron (optional, browser)	Shows a model's input/output names and shapes when something mismatches	netron.app

Pillow, OpenCV, NumPy and ipywidgets come with PaddleOCR / Colab — nothing extra. No Tesseract, no ML Kit, no TensorFlow.

4 · Model choice

One primary path: PaddleOCR PP-OCRv5 mobile — two small models.

- **Detector — PP-OCRv5_mobile_det**, official ONNX (4.8 MB). Finds text boxes, language-independent, used pretrained tonight.
- **Recognizer — en_PP-OCRv5_mobile_rec** (7.5 MB, 85.25% line accuracy on PaddleOCR's English benchmark, 505-symbol English dictionary incl. ₹ € ° μ). **This is the model you fine-tune.**

Base model → fine-tuning → export → optimization → Android runtime

en_PP-OCRv5_mobile_rec_pretrained.pdparams → tools/train.py with config
 configs/rec/PP-OCRv5/multi_language/en_PP-OCRv5_mobile_rec.yaml on your dataset → tools/export_model.py (inference.json / .pdiparams / .yaml) → paddlex --paddle2onnx (inference.onnx) → optional INT8 → Android: ONNX Runtime on CPU with the box + CTC decoding written in Kotlin.

- **Why this one:** small and CPU-fast, official fine-tuning config and pretrained weights, official ONNX route, English-only dictionary (no stray Chinese characters on noisy text), and the same decoding logic for every PP-OCR model.
- **Only fallback:** if training cannot finish, ship the official pretrained en_PP-OCRv5_mobile_rec_onnx — identical Android code, one different file — and label it “pretrained, not fine-tuned”.
- **After tonight:** devanagari_PP-OCRv5_mobile_rec (Hindi + Marathi + English) drops into the same code; PP-OCRv6 (June 2026) is newer but its small recognizer is ~20 MB and multilingual — benchmark it later.

```
# detector, ONNX, ready to use
https://huggingface.co/PaddlePaddle/PP-OCRv5_mobile_det_onnx/resolve/main/inference.onnx
# recognizer baseline / fallback, ONNX
https://huggingface.co/PaddlePaddle/en_PP-OCRv5_mobile_rec_onnx/resolve/main/inference.onnx
# recognizer weights to fine-tune
https://paddle-model-ecology.bj.bcebos.com/paddlex/official_pretrained_model/en_PP-OCRv5_mobile_rec_pretrained.pdparams
```

5 · Dataset / fine-tuning plan

Format (PaddleOCR recognition): folders of *cropped text images* + TXT label files, one line per image: relative/path.jpg<TAB>exact text.

```
vb_ocr_data/
synth/000001.jpg ... (in Google Drive, mounted by Colab)
real/p012_03.jpg ... 10,000 generated crops synth_train.txt -> synth/000001.jpg<TAB>EXP 08/2027
real_train_x10.txt 150-300 crops from 40-60 of your photos
real_val.txt        real training lines repeated 10x (so real data counts as much as synthetic)
                    crops from held-out photos - never trained on, used for every accuracy number
```

- **How much tonight:** 10,000 synthetic crops (5 min to generate) + 150–300 real crops from 40–60 photos, split **80/20 by photo** (never put crops of one photo in both sets).
- **Photograph (same phone, VisionBridge framing):** printed pages and receipts · signs, room and bus numbers · product and food packaging · medicine boxes and strips · small print · curved bottle labels · dim room · slight hand shake · angled shots · glossy glare · screens. Avoid private documents (IDs, bank papers).
- **Labels:** exactly what is visible, same capitals and punctuation, **≤ 25 characters** (the config's max_text_length — longer lines are silently skipped), only dictionary characters. If you cannot read a crop yourself, delete it — never guess.
- **Fast labelling:** the notebook pre-labels every crop with the pretrained model, then shows image + editable text + Save/Next; you only correct mistakes (~5 s per crop).
- **Synthetic data:** Pillow + OpenCV, 15+ free fonts, text from sign / label / packaging / document / menu / price (₹) / date / unit / phone templates plus common English words.

- **Augmentations that matter most:** blur and motion blur · low light + noise · perspective and $\pm 7^\circ$ rotation · glare spot · JPEG 30-90 · low resolution · shadow · light-on-dark.

Training command (Colab, run inside the PaddleOCR folder; the notebook does this for you):

```
python tools/train.py -c configs/rec/PP-OCRv5/multi_language/en_PP-OCRv5_mobile_rec.yaml -o \
  Global.pretrained_model=./pretrained/en_PP-OCRv5_mobile_rec_pretrained.pdparams \
  Global.save_model_dir=./output/vb_en_rec Global.epoch_num=20 "Global.eval_batch_step=[0,200]" \
  Optimizer.lr.learning_rate=0.0002 Optimizer.lr.warmup_epoch=1 \
  Train.dataset.data_dir=/content/vb_ocr_data/ \
  "Train.dataset.label_file_list=['/content/vb_ocr_data/synth_train.txt','/content/vb_ocr_data/real_train_x10.txt']" \
  Eval.dataset.data_dir=/content/vb_ocr_data/ \
  "Eval.dataset.label_file_list=['/content/vb_ocr_data/real_val.txt']" \
  Train.loader.num_workers=2 Eval.loader.num_workers=2 # GPU out of memory? add:
# Train.sampler.first_bs=64 Train.loader.batch_size_per_card=64
```

Output: output/vb_en_rec/best_accuracy.pdparams (+ train.log), roughly 20-45 min on a T4. Run the same tools/eval.py command with the pretrained weights and with your result, and keep the fine-tuned model **only if acc and norm_edit_dis on real_val improve**.

6 · Android integration steps

Inspect these files first (real paths in your repo, under `app/src/main/java/com/example/visionbridge/`):

File	Role — and what to do with it
ui/screens/SmartReadingScreen.kt	CameraX capture, TTS, TalkBack semantics, low-confidence banner — reuse, do not restructure
ui/screens/reading/SmartReadingViewModel.kt	ReadingUiState + Smart Reading's own 0.70 banner threshold — keep the threshold
data/SmartReadingRepository.kt	The one place to switch from cloud to offline (extractText)
api/SmartReadingApi.kt, api/ApiResult.kt	ReadingExtraction(text, confidence, message) — keep the type; do not delete the cloud path
utils/ImageHelper.kt	Shared by MedSafe, Currency, Object Finder, Surroundings, Transport — add, never change
utils/TextToSpeechManager.kt, data/ContextMemoryManager.kt, voice/SmartReadingGroundedQa.kt	Reuse untouched — they already speak and remember the result
androidTest/.../SmartReadingEndToEndTest.kt + androidTest/assets/reading-sample.jpg	Test pattern and a real sample image (contains "PARACETAMOL")

The master-context class names (VoiceManager, LiveAudioEngine, ApiClient...) are **not** on this path — Antigravity must confirm every class by searching the current repo, never by assuming.

Step	Do this
1 Model files	app/src/main/assets/ocr/: ppocrv5_mobile_det.onnx, en_rec_vb_ft.onnx, en_rec_dict.json (character list taken from the shipped model's inference.yml), models.json (source, fine-tuned yes/no, sha256)
2 Gradle	implementation("com.microsoft.onnxruntime:onnxruntime-android:1.30.0") in app/build.gradle.kts. No manifest or permission change (CAMERA already declared)
3 Load once	Singleton OfflineOcrEngine.getInstance(context) (same pattern as SessionManager): one OrtEnvironment + two sessions, 4 intra-op threads, ALL_OPT; warm up when Smart Reading opens — never reload per image
4 Preprocess	Decode JPEG with EXIF/CameraX rotation (long side ≤ 2048). Detector input: long side 960, both sides rounded to a multiple of 32, BGR , (x/255 - [0.485,0.456,0.406]) / [0.229,0.224,0.225], NCHW float32
5 Detect	Probability map → threshold 0.3 → connected regions → minimum-area rectangle → keep box score ≥ 0.6 → expand by $\text{area} \times 2.0 / \text{perimeter}$ → scale back → sort top-to-bottom, left-to-right
6 Recognize	Perspective-crop each box (rotate 90° if height $\geq 1.5 \times$ width) → height 48, keep ratio, BGR, (x/255 - 0.5)/0.5 → batches of 6 padded to the widest (320-3200 px)
7 Decode	CTC greedy: argmax per step, drop index 0 (blank), merge repeats. Labels = ["blank"] + dictionary + [" "] — model classes must equal dictionary size + 2 . Line score = mean of max probabilities
8 Confidence	Drop lines below 0.50; overall confidence = character-weighted mean of all recognized lines; no text → NoText with a next action. Existing 0.70 banner unchanged; MedSafe's gate untouched
9 Threads + memory	All OCR inside withContext(Dispatchers.Default) behind one Mutex, never on the main thread; ARGB_8888 + inSampleSize; recycle bitmaps; close every tensor/result with use {}; catch OutOfMemoryError
10 Lifecycle	Sessions created once, reused, released in onTrimMemory (app hidden) under the same lock and reloaded lazily. Camera binding stays where it is
11 TTS	Repository returns the same ReadingExtraction → existing ViewModel → existing TextToSpeechManager speaks it. No TTS code changes.

7 · Keep existing features safe

- ☐ Change only Smart Reading + the new ocr/ files (plus the Gradle dependency, assets and tests).
- ☐ Reuse SmartReadingScreen's CameraX capture — no second camera session, no preview-frame processing.
- ☐ Reuse TextToSpeechManager, the existing ReadingUiState flow and all TalkBack semantics.
- ☐ Do not touch ImageHelper.optimizeAndEncode, ConfidenceEvaluator, MEDICATION_CONFIDENCE_THRESHOLD or any MedSafe file.
- ☐ Keep SmartReadingApi and the reading-extract Edge Function (simply not called offline) — and no silent cloud fallback.
- ☐ No changes to supabase/, the Web App Node.js backend, Gemini/Live, WebRTC/Volunteer Help, VoiceManager, auth or navigation.
- ☐ No network call, upload or image logging in the OCR path; never log recognized text.
- ☐ Training code lives in ocr-training/, outside the app module; datasets and checkpoints stay git-ignored.

8 · Testing checklist

Group	Run these
Basic	clear printed page · full paragraph · small text (8–10 pt) · text rotated ~15° and 90°
Difficult	dim light · blur / hand shake · glare on glossy packaging · strong angle (perspective) · noisy or low-resolution image
Offline	airplane mode ON + Wi-Fi OFF + mobile data OFF → Read Text still reads aloud; Network Inspector shows no request during the capture
Regression	login · home navigation · voice commands · AI Vision / Currency / Object Finder · Gemini Live + AI Voice Call · Volunteer Help call · MedSafe scan, save, reminder · SOS screen opens (do not trigger it) · TTS everywhere

Measure and write down (never estimate):

Metric	How to get it
Model size	file sizes in assets/ocr + APK size before and after (Build → Analyze APK)
Load time and average OCR time	VB-OfflineOcr logcat timings; median of 10 captures, split into detection / recognition / total
Peak memory	Android Studio Profiler → Memory, while a capture is processed
OCR accuracy	acc and norm_edit_dis on real_val (pretrained vs fine-tuned) + 10 real-photo spot checks

9 · Troubleshooting

Symptom	Most likely fix
Model file not found	The file must sit in app/src/main/assets/ocr/ with the exact name used in code; Rebuild; confirm with APK Analyzer.
ONNX input shape / name error	Open the .onnx in netron.app; use session.inputNames; detector sides must be multiples of 32, recognizer height must be 48, shape [N,3,H,W].
Bitmap preprocessing error	Decode as ARGB_8888, apply rotation before detection, feed BGR float32 in the documented normalization.
OCR output is gibberish	Wrong dictionary or index offsets (blank = 0, space last), RGB instead of BGR, or wrong normalization — compare against the notebook's golden output for reading-sample.jpg.
Text sideways only on the phone	EXIF / CameraX rotation not applied before detection.
Inference too slow	Reuse sessions, 4 intra-op threads, detector long side 736, batch the crops; then try XNNPACK (addXnnpack with 4 threads + setIntraOpNumThreads(1)).
Out of memory	Decode with inSampleSize to ≤ 2048 px, recycle bitmaps, close tensors, one inference at a time.
“ConvInteger not implemented” after quantizing	Plain dynamic quantization does not work on convolutions — use static QDQ INT8, or ship FP32.
App crashes in Smart Reading	Logcat filter “VB-” + AndroidRuntime; usually a closed session, main-thread work or a missing asset.
Works online, fails offline	Something still calls SmartReadingApi / Supabase, or a model is fetched at runtime instead of bundled — search the path for OkHttp, ApiClient, SupabaseClient.
Fine-tuned model worse than baseline	Too few or wrong labels, or too many epochs: fix labels, drop the learning rate to 0.0001, train 10 epochs — or ship the pretrained model and say so.

10 · Final “done” checklist

- ☐ Fine-tuned model exists
- ☐ Model runs locally
- ☐ No network required
- ☐ Smart Reading uses local OCR
- ☐ Existing TTS works
- ☐ Real camera image works
- ☐ Difficult images tested
- ☐ Speed measured
- ☐ Existing VisionBridge features still work
- ☐ APK builds successfully
- ☐ APK tested in airplane mode

11 · Antigravity master prompt — copy everything below into Antigravity

Attach or paste the VisionBridge master context in the same conversation. A clean-copy text file sits next to this PDF:

VisionBridge_Offline_OCR_Antigravity_Prompt.txt — copying from there avoids page footers landing in the middle of the prompt.

You are modifying an existing production-style Android accessibility project called VisionBridge. Do not start by coding. First inspect the repository and determine the actual current implementation.

CONTEXT

- I am a beginner in machine learning and mobile model deployment. Do the coding yourself (do not just explain), and explain each decision in one or two plain sentences.
- The VisionBridge master context document I provided is the baseline, but it may be out of date. The repository is the source of truth. Never assume a class exists or behaves as described - search first. Classes named in the master context (VoiceManager, LiveAudioEngine, GeminiLiveWebSocket, ApiClient, etc.) are NOT assumed to be part of Smart Reading.
- Tonight's goal: add a fine-tuned OCR model that runs completely offline on Android and is used ONLY by the existing Smart Reading ("Read Text") feature: camera image -> offline OCR -> text -> existing accessible UI + TTS.
- Hard rules for the OCR path at runtime: no internet, no Gemini, no Supabase, no Edge Function, no cloud OCR, no image upload, no model download at runtime. Models are bundled in the APK.

- Chosen approach (change it only if inspection proves it cannot work, and ask me first): PaddleOCR PP-OCRv5. Detector = PP-OCRv5_mobile_det (official pretrained ONNX, not fine-tuned tonight). Recognizer = en_PP-OCRv5_mobile_rec fine-tuned on VisionBridge-style data -> ONNX. Runtime = ONNX Runtime for Android, com.microsoft.onnxruntime:onnxruntime-android:1.30.0 (latest on Maven Central on 17 Sep 2026; use 1.29.0 only if 1.30.0 causes a real problem). Pre/post-processing written in Kotlin.
- Facts about this repo to verify (not assume): Kotlin + Jetpack Compose, AGP 9.1.1, Gradle 9.3.1, compileSdk 36, minSdk 24, Java 11, package com.example.visionbridge, no DI framework (singletons use getInstance(context)), no app/src/main/assets folder yet. I should be on branch feature/offline-ocr with a clean checkpoint commit - if not, stop and tell me.
- This is a Windows machine: use PowerShell-friendly commands (.gradlew.bat, adb, curl.exe). Do not rewrite git history and do not commit unless I ask.

PHASE A - INSPECT (read-only, no edits)

1. Inspect the project structure, settings.gradle.kts, both build.gradle.kts files, gradle/libs.versions.toml, app/src/main/AndroidManifest.xml and app/src/main/java/com/example/visionbridge/**.
2. Trace Smart Reading end to end. Expected path (confirm or correct it): MainActivity.kt -> ui/screens/SmartReadingScreen.kt (CameraX Preview + ImageCapture, JPEG bytes + rotationDegrees, TextToSpeechManager, TalkBack semantics, ScreenActionRegistry "reading") -> ui/screens/reading/SmartReadingViewModel.kt (ReadingUiState, LOW_CONFIDENCE_THRESHOLD = 0.70, ContextMemoryManager) -> data/SmartReadingRepository.kt (ImageHelper.optimizeAndEncode -> SmartReadingApi) -> api/SmartReadingApi.kt -> Supabase Edge Function reading-extract (Gemini) -> api/ApiResponse.kt ReadingExtraction(extractedText, confidence, message).
3. Also inspect: utils/ImageHelper.kt and ALL its callers (MedicationRepository, CurrencyReaderViewModel, ObjectFinderViewModel, SurroundingsViewModel, TransportViewModel); utils/TextToSpeechManager.kt; voice/ScreenActionRegistry.kt; voice/SmartReadingGroundedQA.kt; data/ContextMemoryManager.kt; ai/ConfidenceEvaluator.kt; data/MedicationModels.kt; the MedSafe files (ui/screens/medication/*, data/Medication*, api/MedicationApi.kt); res/values/strings.xml, values-hi and values-mr; androidTest/.../SmartReadingEndToEndTest.kt and androidTest/assets/reading-sample.jpg; supabase/functions/reading-extract/index.ts (reference only). The Web App (client/src/modules/Reading) is reference only and is not in this repo.
4. Search the whole repository for existing OCR/ML/inference code or dependencies (ML Kit, TFLite, ONNX, Tesseract, "ocr", "TextRecognition", "Interpreter") and for existing model/assets handling before creating anything.
5. Output an IMPLEMENTATION MAP, then STOP and wait for me to reply "go":
 - (1) Smart Reading entry point
 - (2) screen/UI
 - (3) ViewModel/state holder
 - (4) image capture source
 - (5) current OCR/image-processing code
 - (6) TTS path
 - (7) exact files you will create or modify
 - (8) exact files you will NOT touch, and why
 - (9) whether any other feature, especially MedSafe, shares the Smart Reading OCR path
 - (10) risks and open questions.

PHASE B - DEPLOYMENT DECISION

- Default: fine-tuned lightweight OCR -> ONNX -> ONNX Runtime Android (CPU execution provider). Do not force ONNX if inspection shows another fully offline mobile format is materially simpler or faster for this exact model - but explain the trade-off and ask me first. The final solution must be fully offline either way.
- Do not add ML Kit, TensorFlow Lite, OpenCV, Tesseract or an NDK/C++ layer. Use Android Bitmap/Matrix/Canvas and plain Kotlin for pre/post-processing.
- Record the decision in one paragraph in ocr-training/README.md.

PHASE C - MODEL PREPARATION (training runs in Google Colab on a GPU; you prepare everything)

Create a top-level folder ocr-training/ (NOT inside app/) containing:

- README.md: exact reproducible steps, folder layouts, label format, every command, expected outputs, rough times, and which files come back to the Android project.
- .gitignore: ignore datasets, photos, checkpoints, exports, zips and caches.
- scripts/ (small, commented, runnable in Colab and locally) and VisionBridge_OCR_FineTune.ipynb, a Colab notebook I run top to bottom on a GPU runtime (Runtime -> Change runtime type -> GPU). Notebook cells:
 0. Check the GPU (nvidia-smi). Mount Google Drive. Working dir /content; persistent data in MyDrive/VisionBridgeOCR/ (copy results there after each long step).
 1. Install: pip install paddlepaddle-gpu==3.3.0 -i https://www.paddlepaddle.org.cn/packages/stable/cu126/; verify with python -c "import paddle; paddle.utils.run_check()"; git clone --depth 1 --branch v3.7.0 https://github.com/PaddlePaddle/PaddleOCR.git; pip install -r PaddleOCR/requirements.txt; pip install paddleocr==3.7.0 onnx onnxruntime huggingface_hub; paddlex --install paddle2onnx. If a pinned version is unavailable, use the nearest stable one and print the versions actually installed.
 2. Download: https://paddle-model-ecology.bj.bcebos.com/paddlex/official_pretrained_model/en_PP-OCRv5_mobile_rec_pretrained.pdparams into pretrained/; and with huggingface_hub.snapshot_download the repos PaddlePaddle/PP-OCRv5_mobile_det_onnx and PaddlePaddle/en_PP-OCRv5_mobile_rec_onnx (each has inference.onnx + inference.yml).
 3. Synthetic data (scripts/make_synthetic.py): 10,000 crops, 1-25 characters each, using ONLY characters from PaddleOCR/ppocr/dict/ppocrv5_en_dict.txt plus space (drop anything else and print how many were dropped). Text from templates that match VisionBridge use: signs, room/bus/platform numbers, product and food packaging, medicine-label-style text (for general reading only), printed documents, receipts, menus, prices with the rupee sign, dates such as "EXP 08/2027", units such as "500 mg" and "10 ml", phone numbers, emails - mixed with common English words. Render with Pillow using 15+ open-license fonts (regular, bold, condensed, serif, mono), dark-on-light and light-on-dark, varied backgrounds. Augment with OpenCV/NumPy: Gaussian blur, motion blur, low light + noise, perspective, rotation up to +/-7 degrees, glare spot, shadow, JPEG quality 30-90, downscale-then-upscale. Save synth/*.jpg and synth_train.txt with lines "synth/000001.jpg<TAB>text". Fixed random seed. Show a grid of 20 samples.
 4. Real data (scripts/prelabel_real.py): I upload 40-60 phone photos to MyDrive/VisionBridgeOCR/real_photos/. Run pretrained PaddleOCR (use_doc_orientation_classify=False, use_doc_unwarping=False, use_textline_orientation=False, text_recognition_model_name="en_PP-OCRv5_mobile_rec", text_det_limit_type="max", text_det_limit_side_len=960, text_det_unclip_ratio=2.0) and save every detected text crop plus its predicted text. Keep crops with at most 25 characters; split longer lines at spaces using word boxes (return_word_box) if this PaddleOCR version supports it, otherwise skip those lines and report how many were skipped.
 5. Label review cell (ipywidgets only, no extra installs): show each crop large with an editable text box pre-filled with the prediction, plus Save & Next, Delete (unreadable) and Back buttons. Save progress to Drive after every click. Show this instruction above the widget: "Type exactly what is visible (same capitals and punctuation). If you cannot read it yourself, press Delete. Never guess."
 6. Split (scripts/split_dataset.py): 80/20 by PHOTO - crops from one photo must never be in both sets - into real_train.txt and real_val.txt; create real_train_x10.txt (real_train repeated 10 times for oversampling). Validate that every image exists, every label is 1-25 characters and every character is in the dictionary. Print counts.
 7. Baseline evaluation: python tools/eval.py -c configs/rec/PP-OCRv5/multi_language/en_PP-OCRv5_mobile_rec.yaml with Global.pretrained_model=<pretrained.pdparams>, Eval.dataset.data_dir and "Eval.dataset.label_file_list=['<abs path>/real_val.txt']". Save acc and norm_edit_dis to reports/baseline.json.

8. Fine-tune: `python tools/train.py -c configs/rec/PP-OCrv5/multi_language/en_PP-OCrv5_mobile_rec.yaml -o` with these overrides: `Global.pretrained_model=<pretrained>.pdparams>`, `Global.save_model_dir=./output/vb_en_rec`, `Global.epoch_num=20`, `"Global.eval_batch_step=[0,200]"`, `Optimizer.lr.learning_rate=0.0002`, `Optimizer.lr.warmup_epoch=1`, `Train.dataset.data_dir=<abs data dir>`, `"Train.dataset.label_file_list=['<abs>/synth_train.txt','<abs>/real_train_x10.txt']"`, `Eval.dataset.data_dir=<abs data dir>`, `"Eval.dataset.label_file_list=['<abs>/real_val.txt']"`, `Train.loader.num_workers=2`, `Eval.loader.num_workers=2`. If GPU memory runs out, add `Train.sampler.first_bs=64` `Train.loader.batch_size_per_card=64`. Do NOT change the character dictionary, `max_text_length (25)` or the architecture. Copy `output/vb_en_rec` to Drive when finished.
 9. Fine-tuned evaluation on `real_val.txt` with `output/vb_en_rec/best_accuracy.pdparams -> reports/finetuned.json`. Print a table: baseline vs fine-tuned (acc, norm_edit_dis). If the fine-tuned model is not better, say so plainly and recommend shipping the pretrained recognizer.
 10. Export: `python tools/export_model.py -c <same config> -o` `Global.pretrained_model=./output/vb_en_rec/best_accuracy.pdparams` `Global.save_inference_dir=./export/vb_en_rec_infer` (expect `inference.json`, `inference.pdiparams`, `inference.yml`; do not set `FLAGS_enable_pir_api=0`). Then: `paddlex --paddle2onnx --paddle_model_dir ./export/vb_en_rec_infer --onnx_model_dir ./onnx/vb_en_rec` (expect `inference.onnx`); copy `inference.yml` next to it if it is missing.
 11. ONNX checks (`scripts/check_onnx.py`): `onnx.checker`; print input/output names and shapes; assert `output_classes == len(character_dict) + 2`; run the Paddle inference model (`paddleocr TextRecognition` with `model_dir`) and the ONNX model on every `real_val` crop and report the percentage of identical texts (target $\geq 98\%$).
 12. Optional quantization (`scripts/quantize_rec.py`): `python -m onnxruntime.quantization.preprocess`, then static QDQ INT8 (`quantize_static`, `per_channel=True`) using about 100 real crops as calibration data. Report size, acc and `norm_edit_dis` for FP32 vs INT8. Recommend INT8 only if accuracy drops by 1 point or less; otherwise FP32. Never apply plain dynamic quantization to convolution layers (ConvInteger is not supported on the CPU provider).
 13. Python reference pipeline (`scripts/ocr_reference.py`, `onnxruntime` + `numpy` + `opencv` only) that mirrors the Kotlin logic in Phase D exactly (same constants and steps). Run it on `app/src/androidTest/assets/reading-sample.jpg` (upload it) and 5 of my photos; save `golden/<name>.json` (boxes, texts, scores) and `golden/<name>.png` overlays. Also compare it with PaddleOCR using the same det settings and report differences.
 14. Package `models_out/`: `ppocrv5_mobile_det.onnx`, `en_rec_vb_ft.onnx` (fine-tuned FP32 or INT8), `en_rec_dict.json` (the `character_dict` list from the SHIPPED model's `inference.yml`, read with `yaml.safe_load` - never retyped), `models.json` (model names, source URLs, `fine_tuned` true/false, dataset counts, baseline and fine-tuned metrics, file sizes, sha256 of every file), `golden/`. Zip it to `MyDrive/VisionBridgeOCR/models_out.zip`.
- HANDOFF 1: after creating these files, give me at most 6 short numbered steps for Colab, then continue with Phase D immediately (do not wait for training).

PHASE D - ANDROID MODEL INTEGRATION (build it first with the official pretrained models; swap in the fine-tuned recognizer at Handoff 2)

- Download with `curl.exe -L` from https://huggingface.co/PaddlePaddle/PP-OCrv5_mobile_det_onnx/resolve/main/inference.onnx and https://huggingface.co/PaddlePaddle/en_PP-OCrv5_mobile_rec_onnx/resolve/main/inference.onnx (+ its `inference.yml`) into `app/src/main/assets/ocr/` as `ppocrv5_mobile_det.onnx` and `en_rec_pretrained.onnx`. Generate `en_rec_dict.json` from that `inference.yml` and a `models.json` with `fine_tuned=false`. If downloads fail, ask me to download them.
- Add implementation (`"com.microsoft.onnxruntime:onnxruntime-android:1.30.0"`) to `app/build.gradle.kts`, following the file's existing style. No manifest or permission changes are needed (CAMERA is already declared).
- Create package `com.example.visionbridge.ocr` with small files, for example:
 - `OfflineOcrEngine.kt` - singleton `getInstance(context)` that keeps only `applicationContext`. Lazily creates one `OrtEnvironment` and the `det + rec OrtSessions` from asset bytes (`SessionOptions: setIntraOpNumThreads(min(4, availableCores()), OptLevel.ALL_OPT)`). suspend fun `warmUp()`; suspend fun `read(bitmap): OcrOutput`. All work runs on `Dispatchers.Default` inside one `Mutex`. Registers `ComponentCallbacks2` and releases the sessions in `onTrimMemory (level >= TRIM_MEMORY_UI_HIDDEN)` under the same lock, reloading lazily later. Never closes a session during an inference.
 - `TextDetector.kt` - if the long side is > 960 px scale it down to 960 (keep aspect ratio), then round each side to the nearest multiple of 32 (minimum 32). Channel order BGR; value = $(x/255 - \text{mean})/\text{std}$ with mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225] applied to B, G, R in that order; NCHW float32 in a direct `FloatBuffer`. Read the [1,1,H,W] probability map. DB post-processing: threshold 0.3 -> 8-connected regions -> convex hull (use each row's left/right extremes) -> minimum-area rectangle (rotating calipers) -> skip if short side < 3 -> box score = mean probability inside that rectangle, keep if ≥ 0.6 -> unclip: expand each side by $d = \text{area} \times 2.0 / \text{perimeter}$ -> skip if short side < 5 -> scale back to the original image and clip -> at most 1000 boxes -> reading order (sort by top y, then boxes within 10 px vertically are ordered left to right).
 - `TextRecognizer.kt` - perspective crop of each box with `Matrix.setPolyToPoly` + `Canvas` (bilinear filtering). If crop height $\geq 1.5 \times$ width, rotate it 90 degrees counter-clockwise (as PaddleOCR does); for these vertical crops also try the opposite rotation and keep the higher score. Resize to height 48 keeping aspect ratio; BGR; value = $(x/255 - 0.5)/0.5$. Sort crops by aspect ratio, batch 6 at a time; batch width = $\max(320, \text{ceil}(48 \times \text{widest ratio in the batch}))$, capped at 3200 (squeeze wider crops to 3200); pad with zeros on the right; run; decode.
 - `CtcDecoder.kt` - labels = ["blank"] + dictionary + [" "]. Greedy decoding: argmax per time step, skip index 0, collapse repeats. Line score = mean of the max probabilities of kept steps (0 if none). Throw a clear error if the model's class count != dictionary size + 2.
 - `OcrModels.kt` - `OcrLine(text, score, box)`, `OcrOutput(lines, fullText, confidence, timingsMs, modelId)`.
 - Bitmap decoding - decode the JPEG bytes to `ARGB_8888` with `inSampleSize` so the long side is ≤ 2048 , then apply EXIF rotation, else the `CameraX rotationDegrees`. Put this in the ocr package or add a NEW function to `ImageHelper`. Do NOT change `ImageHelper.optimizeAndEncode`, its constants (1024 px, JPEG 75) or its behaviour - five other features, including `MedSafe`, depend on it.
- Use exactly the same constants and steps as `scripts/ocr_reference.py`.
- Tests: a JVM unit test for `CtcDecoder` and reading order; an instrumented test `OfflineOcrInstrumentedTest` that runs the engine on `androidTest/assets/reading-sample.jpg`, asserts the text contains "PARACETAMOL", logs timings, and (once golden files exist in `androidTest/assets/golden/`) checks text similarity ≥ 0.95 with `golden/reading-sample.json`.
- Close every `OnnxTensor` and `OrtSession.Result` (use `{}.`). Avoid unnecessary bitmap copies; recycle temporary bitmaps. Catch `OutOfMemoryError` and `OrtException` and map them to the existing `ApiError` kinds (IMAGE_PROCESSING / UNKNOWN) with plain-language, actionable messages.
- Log tag "VB-OfflineOcr": model load time, image size, box count, det/rec/total ms, overall confidence and text LENGTH only. Never log recognized text, images or personal data.

PHASE E - SMART READING ONLY

- The only production call site is `SmartReadingRepository.extractText()`. Keep its signature and its return type `ApiResult<ReadingExtraction>`.
- Add a Smart-Reading-only switch (for example object `SmartReadingOcrConfig { const val USE_OFFLINE_OCR = true }`). When true: decode -> `OfflineOcrEngine.read()` -> `ReadingExtraction(text, confidence, message)`, and `SmartReadingApi` is never called. When false: the existing cloud code runs unchanged. There is no automatic cloud fallback.

- Give the repository an application Context with the smallest change (for example SmartReadingViewModel becomes AndroidViewModel(application), still created by viewModel() in SmartReadingScreen). Warm up the engine when that ViewModel is created. SmartReadingEndToEndTest must still compile (it calls SmartReadingRepository() with no arguments; that constructor may keep the cloud behaviour for that backend test).
- Keep SmartReadingApi, the reading-extract Edge Function, SmartReadingEndToEndTest and the existing Screen/ViewModel/TTS behaviour. Do NOT route AI Vision, Hazard Detection, Currency, Object Finder, Surroundings, Transport, VisionBridge Live, AI Voice Call, Volunteer Help or MedSafe through the new engine. Do not redesign any UI.

PHASE F - FALLBACK / SAFETY

- Drop lines with score < 0.50. confidence = character-weighted mean score of ALL non-empty recognized lines (so dropped text lowers it). SmartReadingViewModel's existing 0.70 threshold then shows its existing "hard to read" banner - keep that threshold and wording unchanged.
- If no text is left, return extractedText = null with an actionable message, for example: "I couldn't find readable text. Hold the phone 20 to 30 cm from the text, add more light, and try again." If you add string resources, add them to values, values-hi and values-mr.
- Never invent, autocorrect or complete text. Never add a cloud fallback to the offline path.
- Do NOT change MedSafe, its pipeline, ConfidenceEvaluator, MEDICATION_CONFIDENCE_THRESHOLD or MedSafe's 0.70 confidence gate. This task is only the general Smart Reading offline OCR feature.

PHASE G - ACCESSIBILITY

- Preserve TalkBack labels, focus order, liveRegion announcements, large high-contrast text, the "Reading text. Please wait." speech and all existing buttons. Success, low confidence, no text and every error must be spoken, never only shown. Test once with TalkBack on.

PHASE H - PERFORMANCE

- Load models once, never per image; warm up off the main thread. Run OCR only on the captured photo (the existing single capture), never on preview frames. Keep det at 960 on the long side; if the median total time is above 2 s on my phone, try 736 and report the speed/accuracy trade-off. Batch recognition. Reuse buffers where it is simple. No UI freezes (watch logcat for "Skipped frames"). Report APK size before and after.

PHASE I - OFFLINE VERIFICATION

1. Build and install: `.\gradlew.bat :app:assembleDebug` then `adb install -r app\build\outputs\apk\debug\app-debug.apk`
2. Ask me to turn Airplane mode ON with Wi-Fi OFF and mobile data OFF.
3. Open Read Text, capture a printed page, and confirm the text is read aloud.
4. Watch `adb logcat -s VB-OfflineOcr VB-ReadingRepo VB-SmartReadingApi` and confirm VB-OfflineOcr timings appear and no Smart Reading HTTP/Supabase request happens during the capture (use Android Studio's Network Inspector too if possible).
5. Run the instrumented test while offline: `.\gradlew.bat :app:connectedDebugAndroidTest`
`"-Pandroid.testInstrumentationRunnerArguments.class=com.example.visionbridge.OfflineOcrInstrumentedTest"`
6. Confirm by reading the code that the ocr package and the offline branch of SmartReadingRepository contain no network code.

PHASE J - REGRESSION PROTECTION

- `.\gradlew.bat :app:assembleDebug` and `.\gradlew.bat :app:testDebugUnitTest` must pass. Report pre-existing failures separately; do not "fix" unrelated code.
- Show git diff --stat, explain every changed file, and revert anything unrelated.
- With network ON, check on the phone: Smart Reading, TTS, login, home navigation, voice commands, and that AI Vision / Currency / Object Finder, VisionBridge Live, Volunteer Help and MedSafe still open and work.
- Confirm no new network dependency was introduced.

HANDOFF 2 - SWAP IN THE FINE-TUNED MODEL

When I give you models_out.zip: verify every sha256 against models.json; replace `en_rec_pretrained.onnx` with `en_rec_vb_ft.onnx` (and the dictionary if it differs); copy golden files to `androidTest/assets/golden/`; set `fine_tuned=true` only if models.json shows the fine-tuned model beat the baseline; repeat Phases I and J; compare phone output with `golden/*.json`; measure again.

PHASE K - IF TRAINING OR EXPORT CANNOT FINISH TONIGHT

- Identify the exact blocker (command, error message, log line) and fix it if reasonably possible.
- Use the fastest legitimate path: ship the official pretrained recognizer through the same Android code.
- Never fake training, metrics or test results. In every report clearly separate: pretrained model / fine-tuned model / exported ONNX / quantized model / integrated in the app.

PHASE L - FINAL REPORT (in chat and saved as ocr-training/REPORT.md)

Report, in this order: 1. Files inspected 2. Files created 3. Files modified 4. Dependencies added 5. Models used (det + rec, with sources) 6. Training actually performed (data counts, epochs, time, baseline vs fine-tuned acc and norm_edit_dis) 7. Model format (FP32 or INT8 ONNX) 8. Model sizes and APK size change 9. Average and median OCR time on the device (det / rec / total, 10 runs) 10. Devices tested (model, Android version) 11. Offline test result 12. OCR test results (easy and difficult images) 13. Existing-feature regression results 14. Known limitations (for example: English only, detector not fine-tuned, upside-down text) 15. Exact next action if anything is incomplete.

Never say "complete" just because Gradle compiled. Mark anything not verified on a real device as NOT VERIFIED.

CODING STYLE

- I am a beginner: keep the code understandable, files small and names clear. Add comments only where they explain non-obvious ML/inference code.
- Do not over-engineer, add unnecessary libraries, create duplicate services, rewrite working architecture or change unrelated features. Follow existing patterns (ApiResponse/ApiError, getInstance(context), "VB-" log tags).
- Inspect first. Search before creating. Build and test after each meaningful change. Fix real errors instead of stopping at the first failure.
- If a choice is ambiguous, pick the smallest practical option that fits this project and explain it afterwards.
- Never invent missing project details. Never claim something was tested if it was not.